

SHERYIANS CODING SCHOOL

Python Complete Notes

From your very first line of code to advanced OOP — everything
you need, in one place.



Watch alongside! This book is designed to work hand-in-hand
with the video series on **Sheryians AI** on YouTube. Every concept
here has a matching video explanation.

What is Python?

Python in Simple Words

Python is a **high-level, general-purpose programming language** created by **Guido van Rossum** and first released in **1991**. The name comes not from the snake, but from the British comedy show *Monty Python's Flying Circus* — because Guido wanted the language to be fun to use.

Python is designed to be **simple to read and write**. Its syntax looks almost like plain English, which is why it's the most beginner-friendly language in the world — and at the same time, powerful enough to run Instagram, YouTube, and NASA systems.

Easy to Learn

No semicolons, no curly braces, no type declarations. Just clean, readable code that looks like English.

Incredibly Powerful

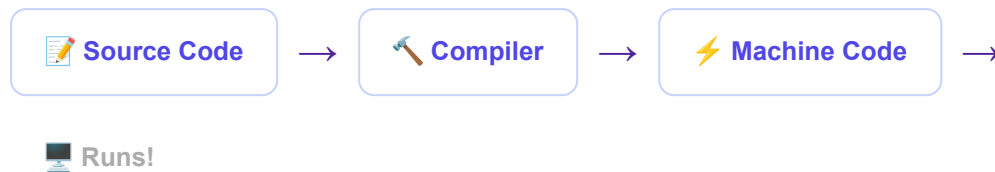
Used in AI, web development, data science, automation, game dev, cybersecurity and more.

How Does Code Actually Run?

Before Python makes sense, you need to understand how computers run code. There are two main approaches — **Compiled** and **Interpreted**.

Compiled Languages — Build First, Run Later

Think of it like **translating an entire book** from Hindi to English before giving it to someone. The whole translation happens upfront — this is called *compiling*. Once compiled, the program runs extremely fast because the computer already has the translation ready.



Examples: C, C++, Rust, Go

Interpreted Languages — Translate Line by Line

Think of it like a **live interpreter at a conference** who translates speech sentence by sentence as the speaker talks. There's no pre-translation — each line is read, translated, and executed one at a time.



Examples: **Python**, JavaScript, Ruby



WHERE DOES PYTHON FIT?

Python is an

INTERPRETED LANGUAGE

. When you run a Python file, the Python interpreter reads your code line by line and executes it immediately. That's why errors show up one at a time — it stops at the first problem it hits.

Compiled vs Interpreted — Side by Side

Feature	Compiled	Interpreted (Python)
---------	----------	----------------------

Translation	All at once before running	Line by line while running
Speed	⚡ Faster execution	🐢 Slightly slower
Error Detection	All errors found before running	Stops at the first error
Portability	Platform-specific binary	Runs anywhere Python is installed
Development Speed	Slower to write & test	✅ Fast to write & test
Examples	C, C++, Rust, Go	Python, JavaScript, Ruby



FUN FACT:

Python actually compiles your code to *bytecode* (`.pyc` files) first, then interprets that bytecode using the Python Virtual Machine (PVM). So technically it's a bit of both — but we call it interpreted because you never see or manage the compiled step.

Why Python? Where is it Used?

Python's simplicity and the massive ecosystem of libraries make it useful in almost every field of technology. Here's where Python truly shines:



AI & Machine Learning

Libraries like TensorFlow, PyTorch, and scikit-learn make Python the #1 language for AI. ChatGPT, Gemini, and most modern AI tools are built with Python.



Data Science & Analytics

Pandas, NumPy, Matplotlib — companies use Python to analyse millions of rows of data and turn them into insights and charts.



Web Development

Django and Flask are powerful Python frameworks. Instagram, Pinterest, and Spotify's backend all run on Python.



Automation & Scripting

Automate boring tasks — rename 1000 files, scrape websites, send automated emails, schedule tasks. Python makes it simple.



Cybersecurity

Python is the go-to language for ethical hacking and penetration testing. Tools like Metasploit and many security scripts are Python-based.



Game Development

Pygame lets you build 2D games with Python. It's a great way to practise programming while building something fun.

✨ Key Features of Python

🔧 Clean Syntax

Reads like English. Indentation is mandatory, making all Python code look consistent.



Free & Open Source

Python is completely free to download, use, and distribute — even commercially.



Huge Library Ecosystem

PyPI has over 400,000 packages. Whatever you want to build, there's probably already a library for it.



Cross-Platform

Write once, run anywhere — Windows, macOS, Linux. Same Python code works on all of them.

Great Community

Millions of developers, endless tutorials, Stack Overflow answers, and active forums.

Multi-Paradigm

Supports functional, procedural, and object-oriented programming styles.

Python vs Other Languages — A Quick Taste

Let's print "Hello, World!" in three different languages to see how clean Python really is:

Java

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
    }  
}
```

5 lines just to print one thing 😓

Python

```
print("Hello, World!")
```

1 line. That's it. ✅



YOU'RE ALL SET!

Now that you know what Python is, where it came from, how it runs, and where it's used — let's get it installed and write some actual code. Head to Chapter 01!

CHAPTER 01

Installation

01

Downloading Python

Open any browser → go to **python.org** → download Python for your operating system.

Run the installer. This gives you the **Python Virtual Machine** which converts your code into byte code that your computer can run.



IMPORTANT

Check "Add Python to PATH" during installation on Windows — otherwise your terminal won't find Python!

Downloading an IDE

An **IDE (Integrated Development Environment)** is where you write and run your code. Popular choices are VS Code, PyCharm, and Jupyter — but we'll use **VS Code** throughout this book.

Setting Up VS Code

Open VS Code → go to the **Extensions** panel (Ctrl+Shift+X)

Search for and install: **Python** (by Microsoft) and **Code Runner**

Create a new file ending in `.py` and you're ready to go!

CHAPTER 02

Comments & Variables

02

Comments

Comments are notes you write in your code for yourself (or other developers). Python completely ignores them — they don't affect how the program runs.

```
# This is a single-line comment
```

```
"""
```

```
This is a multiline comment  
written using a docstring  
"""
```



Python doesn't have a true multiline comment syntax. We "borrow" the triple-quote string `""" . . . """` for this purpose.

Variables

Think of a variable as a **labelled box** — you put a value inside and refer to it by the label whenever you need it.

```
name = "Akarsh"  
age = 20  
city = "Indore"
```



RULES — THESE WILL CAUSE ERRORS:

- ❌ `1name = "x"` → can't start with a number
- ❌ `my name = "x"` → no spaces allowed
- ❌ `my-name = "x"` → no special characters (except underscore)

Naming Conventions

```
camelCase → myVariableName  
PascalCase → MyVariableName  
snake_case → my_variable_name # ✅ Python prefers this
```

CHAPTER 03

Data Types

03

What are Data Types?

Every value in Python has a **type** that tells Python what kind of data it is and what you can do with it. You don't need to declare types — Python figures it out automatically.

The Main Data Types

`int`

Whole numbers: 1, 42, -7

float

Decimal numbers: 3.14, -0.5

complex

Real + imaginary: 3+4j

str

Text in quotes: "hello"

bool

Only two values: True or False

NoneType

Represents nothing: None

Checking the Type

```
print(type(42))          # <class 'int'>
print(type(3.14))        # <class 'float'>
print(type("hello"))     # <class 'str'>
print(type(True))        # <class 'bool'>
```

How Strings Work Internally

Each character in a string is stored with its own **Unicode number**. That's why strings use more memory than integers.

```
ord("A")    # → 65   (Unicode of A)
chr(65)     # → "A"  (Character from Unicode)
```

String Indexing

Every character in a string has a position number called an **index**. Positive indexes count from the left (starting at 0), negative from the right (starting at -1).

```
a = "Hello"
#   H e l l o
#   0 1 2 3 4   ← positive
#  -5 -4 -3 -2 -1 ← negative

print(a[0])    # H
print(a[-1])   # o
```

String Slicing

Slicing cuts out a piece of a string. Syntax: `string[start : stop : step]` — note that `stop` index is **excluded**.

```
a = "hello"
print(a[1:4])    # ell  (index 1,2,3 - 4 excluded)
print(a[::-1])   # olleh (reversed!)
```

Type Conversion

You can convert a value from one type to another using these built-in functions:

int()

→ whole number

float()

→ decimal number

str()

→ text

bool()

→ True or False

Implicit (Automatic)

Python converts automatically when needed.

```
a = 12
print(a / 2)  # 6.0
# int ÷ int → float!
```

Explicit (Manual)

You tell Python to convert.

```
a = 12
a = str(a)
print(a)  # "12"
```

The 7 Falsy Values

Everything converts to `True` with `bool()` — *except* these 7 values which become `False`:

0

0.0

`False`

""

[]

{}

()

CHAPTER 05

Input, Output & Operators

05

Output — print()

```
name = "Akarsh"
age  = 20

print("Hello!")           # basic
print(f"My name is {name}") # f-string
print("Name:", name, "Age:", age) # multiple values
```

Input — input()

REMEMBER:



`input()` always returns a

STRING

. If you need a number, convert it manually with `int()` or `float()`.

```
name = input("What is your name? ")
age  = int(input("How old are you? "))

print(f"Hello {name}, you are {age} years old!")
```

Arithmetic Operators

Operator	Name	Example	Result
+	Addition	10 + 3	13
-	Subtraction	10 - 3	7
*	Multiplication	10 * 3	30
/	Division	10 / 3	3.333...
//	Floor Division	10 // 3	3
%	Modulus (remainder)	10 % 3	1
**	Exponentiation	2 ** 8	256

Comparison Operators

Always return `True` or `False`.

Operator	Meaning	Example	Result
----------	---------	---------	--------

<code>==</code>	Equal to	<code>5 == 5</code>	True
<code>!=</code>	Not equal to	<code>5 != 3</code>	True
<code>></code>	Greater than	<code>5 > 3</code>	True
<code><</code>	Less than	<code>5 < 3</code>	False
<code>>=</code>	Greater or equal	<code>5 >= 5</code>	True
<code><=</code>	Less or equal	<code>3 <= 5</code>	True

Logical Operators

Operator	Returns True when...	Example
<code>and</code>	Both conditions are True	<code>age > 18 and has_id == True</code>
<code>or</code>	At least one condition is True	<code>is_admin or is_staff</code>
<code>not</code>	Reverses the boolean	<code>not is_banned</code>

Assignment Operators

Operator	Meaning	Equivalent to
<code>+=</code>	Add and assign	<code>x = x + n</code>
<code>-=</code>	Subtract and assign	<code>x = x - n</code>
<code>*=</code>	Multiply and assign	<code>x = x * n</code>
<code>/=</code>	Divide and assign	<code>x = x / n</code>

//=	Floor divide and assign	x = x // n
%=	Modulus and assign	x = x % n
**=	Power and assign	x = x ** n

CHAPTER 06

Conditional Statements

06

Making Decisions in Code

Real programs don't run the same code every time — they **make decisions**. Conditional statements let your program choose what to do based on a condition. That's why they're also called *control flow statements*.

```
if condition:
    # runs when condition is True
elif another_condition:
    # runs if the above was False, this is True
else:
    # runs when nothing above was True
```

Types at a Glance

Statement	When to use it
<code>if</code>	You have one condition to check

if-else

Two paths — True or False

if-elif-else

Multiple conditions checked one by one



Practice Questions

Q1

Accept two numbers and print the greatest between them.

Input: 14, 7

Output: 14 is greater

Q2

Accept gender from user and print a greeting message.

Input: M

Output: Good Morning Sir

Q3

Accept an integer and check if it is even or odd.

Input: 9

Output: 9 is Odd

Q4

Accept name and age — check if the user is a valid voter (18+).

Input: Shery, 20

Output: Hello Shery, you are a valid voter ✓

Q5

Accept a year and check if it is a leap year.

Input: 2024

Output: 2024 is a leap year ✓

Q6 — Temperature Ladder

Accept temperature in °C and print a description.

Input: -5

Freezing Cold 🥶

Input: 25

Pleasant 😊

Input: 45

Very Hot 🔥

CHAPTER 07

Loops

07

Why Loops?

Imagine printing "Hello" 100 times. Without loops: 100 lines of code. With a loop: just 2 lines. Loops let you **repeat a block of code** without rewriting it.

Python has 2 types of loops: **for** and **while**.

The Bucket Analogy 🪣



Two buckets, one mug — which loop do you use?

FOR loop — known iterations

Transfer exactly **4 mugs**. You know the count → use `for`.

WHILE loop — known condition

Transfer **until bucket is empty**. You don't know the count, but you know when to stop → use `while`.

CHAPTER 08

For Loop

08

The `range()` Function

`range()` generates a sequence of numbers. Think of it as saying "count from here to there".

```
range(stop)           # 0 up to stop-1
range(start, stop)    # start up to stop-1
range(start, stop, step) # start, jumping by step

list(range(5))         # [0, 1, 2, 3, 4]
list(range(1, 6))      # [1, 2, 3, 4, 5]
list(range(0, 10, 2))  # [0, 2, 4, 6, 8]
```

For Loop with Numbers

```
for i in range(1, 6):
    print(i)
```

```
# Output: 1 2 3 4 5
```

For Loop with Strings

```
name = "hello"

# Method 1 - via index
for i in range(len(name)):
    print(name[i])

# Method 2 - direct (simpler!)
for char in name:
    print(char)
```

break, continue & else — The Traffic Light Analogy 🚦

🚦 **Imagine you're driving through 10 traffic signals on your way home**

Each signal = one loop iteration. Here's what each keyword does:



break

You spot an accident ahead — you **immediately stop** and take a U-turn. Loop ends completely.



continue

One signal is broken — you **skip it** and keep driving to the next one. Loop skips this iteration.



else

You crossed all signals with no problems — you reached home safely. Runs only when loop finishes **without a break**.

```
for i in range(1, 11):
    if i == 5:
        break          # stop at signal 5 (accident!)
    if i == 3:
        continue       # skip signal 3 (broken light)
    print(f"Signal {i} — passed ✅")
else:
    print("Reached home safely! 🏠") # only if no break
```

For Loop Questions

Q1

Print "Hello World" n times.

Input: 3

Hello World × 3 lines

Q2

Print natural numbers from 1 to n.

Input: 5

1 2 3 4 5

Q3

Reverse for loop — print n down to 1.

Input: 5

5 4 3 2 1

Q4

Print the multiplication table of a number.

Input: 5

5×1=5, 5×2=10 ... 5×10=50

Q5

Sum of first n natural numbers.

Input: 5

Sum = 15

Q6

Factorial of a number.

Input: 5

5! = 120

Q7

Print sum of all even and odd numbers in a range separately.

Input: 1 to 10

Even sum = 30, Odd sum = 25

Q8

Print all factors of a number.

Input: 12

1 2 3 4 6 12

Q9

Check if a number is perfect (sum of factors = the number itself).

Input: 6

6 is a Perfect Number ✓ (1+2+3=6)

Q10

Check if a number is prime.

Input: 17

17 is Prime ✓

Input: 9

9 is NOT Prime ✗

Q11

Reverse a string without using built-in functions.

Input: "Python"

nohtyP

Q12

Check if a string is a palindrome.

Input: "racecar"

Palindrome ✓

Input: "hello"

Not a palindrome ✗

Q13

Count letters, digits, and special symbols in a string.

Input: "P@#yn26at^&i5ve"

Chars=8, Digits=3, Symbols=4

CHAPTER 09

While Loop

09

While Loop

The while loop keeps running **as long as a condition is True**. You use it when you don't know how many times you'll need to repeat.

```
count = 1
while count <= 5:
    print(count)
    count += 1
```

Output: 1 2 3 4 5



INFINITE LOOP DANGER!

Always make sure your condition will eventually become False — otherwise your program runs forever!



While loops also support

BREAK

,

CONTINUE

, and

ELSE

exactly like for loops.



While Loop Questions

Q1

Separate each digit of a number and print on a new line.

Input: 1234

4 → 3 → 2 → 1

Q2

Accept a number and print its reverse.

Input: 12345

54321

Q3

Check if a number is palindromic (equal to its reverse).

Input: 121

Palindrome ✓

Input: 123

Not Palindrome ✗

Q4

Build a number guessing game — computer picks a random number, user keeps guessing until correct.

Guess: 50 → Too low! Guess: 75 → Too high! Guess: 63 → 🎉 Correct!

Functions



What are Functions?

A function is a **reusable block of code** with a name. Instead of writing the same logic 10 times, you write it once as a function and call it 10 times.

```
def greet():  
    print("Hello, welcome to Python!")  
  
greet()    # ← this is calling the function
```

Parameters & Arguments

Parameter

The variable name in the function definition. Like a placeholder.

```
def greet(name):    # ← parameter  
    ...
```

Argument

The actual value you pass when calling the function.

```
greet("Alice")    # ← argument
```

Types of Arguments

```
# 1. Positional - order matters
def add(a, b):
    return a + b
add(5, 3)          # → 8

# 2. Default - works even without passing a value
def greet(name="Guest"):
    print(f"Hello {name}")
greet()             # Hello Guest
greet("Akarsh")     # Hello Akarsh

# 3. Keyword - pass in any order
def info(name, age):
    print(f"{name} is {age}")
info(age=25, name="Akarsh") # order doesn't matter
```

CHAPTER 11

Data Structures

11

The 4 Built-in Data Structures

When you need to store *multiple values* in one variable, you use a data structure. Python gives you 4 ready to use:

Structure	Ordered?	Mutable?	Duplicates?	Access by
List	✓ Yes	✓ Yes	✓ Yes	Index
Tuple	✓ Yes	✗ No	✓ Yes	Index

Set

✗ No

✓ Yes

✗ No

Methods

Dictionary

✓ Yes

✓ Yes

Keys: ✗

Key

CHAPTER 12

List

12

Creating and Accessing Lists

```
fruits = ["apple", "banana", "mango"]

print(fruits[0])    # apple
print(fruits[-1])   # mango
print(fruits[0:2])  # ['apple', 'banana']

fruits[1] = "grape" # mutation – lists allow this!
```

Key List Methods

```
lst = [3, 1, 4, 1, 5]

lst.append(9)      # [3,1,4,1,5,9] – add to end
lst.insert(0, 0)   # [0,3,1,4,1,5,9] – insert at index
lst.remove(1)      # removes first 1
lst.pop()          # removes last element
lst.sort()         # sort ascending
```

```
lst.reverse()      # reverse in place
len(lst)           # number of items
```

List Questions

Q1

Print all positive and negative elements separately.

Input: [3, -1, 4, -5, 9]

Positive: [3,4,9] Negative: [-1,-5]

Q2

Find the mean (average) of all list elements.

Input: [10, 20, 30, 40]

Mean = 25.0

Q3

Find the greatest element and print its index.

Input: [4, 8, 2, 9, 1]

Greatest = 9 at index 3

Q4

Find the second greatest element.

Input: [4, 8, 2, 9, 1]

Second greatest = 8

Q5

Check if the list is already sorted.

Input: [1, 3, 5, 7]

List is sorted 

Input: [3, 1, 4]

Not sorted 

Tuple — The Immutable List

A tuple is exactly like a list, except **you cannot change it** once created. Use tuples for data that should stay constant — like days of the week, coordinates, or config values.

```
days = ("Mon", "Tue", "Wed")

print(days[0])    # Mon
days[0] = "X"    # ❌ TypeError - tuples are immutable
```

Tuple Methods (Only 2!)

```
t = (1, 2, 3, 2, 1)
t.index(2)    # → 1  (first position of 2)
t.count(2)    # → 2  (2 appears twice)
```

Set — Unique Values Only

A set automatically removes duplicates and has no guaranteed order. Great for checking membership and performing math-style set operations.

```
s = {1, 2, 2, 3, 3, 3}
print(s)    # {1, 2, 3} - duplicates removed!
```

Set Operations

```
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}

a | b    # Union          → {1,2,3,4,5,6}
a & b    # Intersection   → {3,4}
a - b    # Difference     → {1,2}
a ^ b    # Symmetric diff → {1,2,5,6}
```

CHAPTER 15

Dictionary

15

Key-Value Storage

A dictionary stores data as **key: value pairs** — like a real dictionary where you look up a word (key) to find its meaning (value).

```
person = {"name": "Akarsh", "age": 20, "city": "Indore"}

# Read
```

```
print(person["name"])          # Akarsh

# Update
person["age"] = 21

# Add new key
person["course"] = "Python"

# Delete
del person["city"]

# Traverse
for key, val in person.items():
    print(key, "→", val)
```

Dictionary Questions

Q1

Merge two dictionaries into one.

```
d1={a:1}, d2={b:2}
```

```
{a:1, b:2}
```

Q2

Sum all values in a dictionary.

```
{"a":10,"b":20,"c":30}
```

```
Sum = 60
```

Q3

Count the frequency of each element in a list using a dictionary.

```
["a","b","a","c","b","a"]
```

```
{"a":3,"b":2,"c":1}
```

Q4

Combine two dicts, adding values for common keys.

```
d1={a:5,b:3}, d2={b:4,c:2}
```

```
{a:5, b:7, c:2}
```

CHAPTER 16

Exception Handling

16

Errors vs Exceptions

✗ Errors (unfixable)

SyntaxError — wrong syntax

IndentationError — bad spacing

TabError — mixing tabs/spaces

✓ Exceptions (handleable!)

ZeroDivisionError

TypeError

ValueError

FileNotFoundError

Handling Exceptions

```
try:
    result = 10 / 0
except ZeroDivisionError:
```

```
    print("Can't divide by zero!")
else:
    print("Success:", result)
finally:
    print("This always runs.")
```

Keyword	Purpose
try	Wrap the risky code
except	Handle the exception
else	Runs only if no exception occurred
finally	Always runs — good for cleanup
raise	Manually throw your own exception

CHAPTER 17

File Handling

17

File Modes

Mode	Meaning	Creates file?
'r'	Read only	 (must exist)
'w'	Write (overwrites!)	
'a'	Append to end	

'x'

Create (fails if exists)



Reading & Writing

```
# Writing
with open("notes.txt", "w") as f:
    f.write("Hello from Python!")

# Reading
with open("notes.txt", "r") as f:
    content = f.read()
    print(content)

# Appending
with open("notes.txt", "a") as f:
    f.write("\nAdded a new line!")
```



Always use the
WITH

statement — it automatically closes the file for you, even if an error occurs.

CHAPTER 18

OOP in Python

18

Three Ways to Write the Same Program

```

# Imperative - simple but not reusable
a, b = 5, 3
print(a + b)

# Functional - reusable
def add(a, b): return a + b

# Object Oriented - scalable & organised
class Calculator:
    def add(self, a, b): return a + b

calc = Calculator()
print(calc.add(5, 3))

```



OOP key concepts:

CLASSES · OBJECTS · ENCAPSULATION · INHERITANCE · POLYMORPHISM · ABSTRACTION

CHAPTER 19

Classes

19

Class = Blueprint

A class is a **blueprint** — like an architect's plan for a house. The plan itself isn't a house, but you can build many houses from it. Each house is an **object**.

```

class Dog:
    species = "Canis lupus"          # ← Attribute

    def bark(self):                  # ← Method
        print("Woof!")

```

```
my_dog = Dog()           # create object
print(my_dog.species)    # Canis lupus
my_dog.bark()            # Woof!
```

CHAPTER 20

Objects

20

Objects = Instances of a Class



Think of a

BAG FACTORY

. The factory has a blueprint (class) that needs material, zips, and pockets.
Reebok and Campus both use this blueprint but provide their own specifications — they become two different

OBJECTS

.

```
class Bag:
    def __init__(self, material, zips):
        self.material = material
        self.zips = zips

reebok = Bag("leather", 3)    # object 1
campus = Bag("nylon", 2)     # object 2

print(reebok.material)      # leather
print(campus.material)      # nylon
```

Constructor

`__init__` — The Constructor

The constructor is a special method that **runs automatically** the moment you create an object. You use it to set up the object's initial data.

`self` is the object itself — it's how the method knows which object's data to set.

```
class Student:
    def __init__(self, name, grade):
        self.name = name    # stored on THIS object
        self.grade = grade

s1 = Student("Akarsh", "A")
s2 = Student("Shery", "B")

print(s1.name)    # Akarsh
print(s2.name)    # Shery
```

Attributes & Methods

Attributes

Class Attribute

Shared by *all* objects of the class.

```
class Dog:
    species = "Canis" # class
```

Instance Attribute

Unique to *each* object.

```
def __init__(self, name):
    self.name = name # instance
```

Methods

```
class Example:
    count = 0

    def instance_method(self):          # needs self
        return "Works with object"

    @classmethod
    def class_method(cls):             # needs cls
        return cls.count

    @staticmethod
    def static_method():               # needs neither
        return "Just a helper function"
```



Child Inherits from Parent

Just like children inherit traits from parents, a child class automatically gets all attributes and methods of the parent class.

```
class Animal:
    def breathe(self):
        print("Breathing...")

class Dog(Animal):      # Dog inherits from Animal
    def bark(self):
        print("Woof!")

d = Dog()
d.breathe()    # inherited from Animal ✓
d.bark()       # Dog's own method ✓
```

Types of Inheritance

```
# Single - one parent
class Child(Parent): ...

# Multiple - two parents
class Child(Parent1, Parent2): ...
# MRO: Parent1's methods take priority

# Multilevel - grandparent → parent → child
class Child(Parent):
    def __init__(self, name, grade):
```

```
super().__init__(name) # call Parent's __init__
self.grade = grade
```

CHAPTER 24

Polymorphism

24

Same Name, Different Behaviour

Polymorphism = "many forms". The same method name behaves differently depending on which object calls it.

```
class Dog:
    def speak(self): print("Woof! 🐶")

class Cat:
    def speak(self): print("Meow! 🐱")

class Duck:
    def speak(self): print("Quack! 🦆")

# Same function call - different results!
for animal in [Dog(), Cat(), Duck()]:
    animal.speak()
```



DUCK TYPING:

"If it walks like a duck and quacks like a duck — it's a duck." Python doesn't care about the *type* of object, only whether it has the method you're calling.

Encapsulation

Hiding Internal Details

Encapsulation means keeping data **safe inside a class** and only exposing what's necessary. Think of it like a medicine capsule — the drug is inside, protected.

```
class BankAccount:
    def __init__(self):
        self.owner      = "Akarsh"      # public
        self._balance   = 1000          # protected (convention)
        self.__pin       = 1234         # private (enforced)

    def get_balance(self):               # safe accessor
        return self._balance

acc = BankAccount()
print(acc.owner)      # ✓ Akarsh
print(acc._balance)   # ⚠ works but bad practice
print(acc.__pin)      # ✗ AttributeError
```

Abstraction

Hide Complexity, Show Simplicity

Abstraction means showing only what the user needs to see, and hiding how it actually works. Like a TV remote — you press a button, you don't need to know the electronics inside.

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):      # defined, not implemented
        pass

class Circle(Shape):
    def __init__(self, r): self.r = r
    def area(self): return 3.14 * self.r ** 2

c = Circle(5)
print(c.area())      # 78.5
```

CHAPTER 27

Dunder Methods

27

Magic Methods

Dunder (double underscore) methods let your objects behave like built-in Python types. They're called automatically when you use operators or built-in functions.

```
class Vector:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __str__(self):      # called by print()
        return f"({self.x}, {self.y})"
```

```

def __add__(self, other):    # called by +
    return Vector(self.x + other.x, self.y + other.y)

def __len__(self):          # called by len()
    return 2

v1 = Vector(1, 2)
v2 = Vector(3, 4)
print(v1)                   # (1, 2)
print(v1 + v2)              # (4, 6)
print(len(v1))              # 2

```

CHAPTER 28

Advanced Topics

28

Decorators

A decorator wraps a function to add extra behaviour without modifying its code. Think of it as gift wrapping — the gift (function) is still the same, but it now has a wrapper around it.

```

def timer(func):
    def wrapper():
        print("🕒 Starting...")
        func()
        print("✅ Done!")
    return wrapper

@timer
def greet():
    print("Hello!")

```

```
greet()
# 🕒 Starting...
# Hello!
# ✅ Done!
```

*args and **kwargs

When you don't know how many arguments a function will receive, use `*args` (for positional) and `**kwargs` (for keyword).

```
def total(*args):          # args is a tuple
    return sum(args)

print(total(1, 2, 3, 4))   # 10
print(total(10, 20))      # 30

def profile(**kwargs):     # kwargs is a dict
    for k, v in kwargs.items():
        print(f"{k}: {v}")

profile(name="Akarsh", age=20)
```

Comprehensions — One-liners

```
# List comprehension
squares = [x**2 for x in range(5)]      # [0,1,4,9,16]
evens    = [x for x in range(10) if x%2==0] # [0,2,4,6,8]

# Dict comprehension
squared = {x: x**2 for x in range(5)}   # {0:0,1:1,2:4...}
```

```
# Set comprehension
unique = {x%3 for x in range(10)}          # {0,1,2}
```

Lambda Functions

```
# Lambda = anonymous one-line function
square = lambda x: x ** 2
add     = lambda a, b: a + b
check   = lambda x: "even" if x%2==0 else "odd"

print(square(5))          # 25
print(add(3, 7))          # 10
print(check(4))           # even
```

map(), filter(), zip()

```
nums = [1, 2, 3, 4, 5]

# map - transform every item
doubled = list(map(lambda x: x*2, nums))  # [2,4,6,8,10]

# filter - keep items that pass the test
evens = list(filter(lambda x: x%2==0, nums)) # [2,4]

# zip - combine two lists into pairs
names  = ["A", "B", "C"]
scores = [90, 85, 78]
pairs  = list(zip(names, scores))  # [('A',90), ('B',85), ...]
```

Modules & Packages

```
# Built-in modules
import math
import random
from datetime import datetime

print(math.sqrt(16))           # 4.0
print(random.randint(1, 100))  # random number
print(datetime.now())          # current date/time

# Third-party (install with pip)
# pip install numpy pandas matplotlib
```

Advanced Challenge Questions

Pattern 1

Print a right-angle triangle with stars.

Input: 4

* / ** / *** / ****

Pattern 2

Print a mirrored right-angle triangle.

Input: 4

**** / *** / ** / *

Pattern 3

Print a centred diamond with stars.

Input: 5

Diamond shape

Q1 — Strong Number

A number is strong if sum of factorials of its digits equals the number.

Input: 145

Strong  (1!+4!+5! = 145)

Q2 — Prime Range

Print all prime numbers between two numbers.

Input: 10, 30

11 13 17 19 23 29

Q3 — Most Frequent

Find which element occurred most in a list.

Input: [1,3,3,2,1,3,4]

3 (appeared 3 times)

A MESSAGE FROM THE CREATOR

"We'll be learning all of this and so much more on this channel. I, **Akarsh Vyas**, as a creator and representative of Sheryians AI, would like to sincerely thank each and every one of you who stayed with us till the end. You are truly precious to us. Much love! 

SHERYIANS CODING SCHOOL